Quadcopter

Andrew Lu, Priyansh Mehta

June 2025



Abstract

In this paper, we will detail our process for researching, designing, building, and coding a quadcopter from scratch without any prior experience in drone creation. First, the choice of materials and tools used will be outlined. Next, we will explore the various design choices made in the CAD models for the structure of our quadcopter as well as the wiring for all of the electrical components, including issues we faced and how we solved them. Then, we detail the building process, which includes soldering, drilling, and additional design considerations. Finally, we explore the usage of PID tuning and its implementation into Arduino software in order to stabilize the flight of our quadcopter, but we unfortunately ran into some issues when applying enough throttle to our quadcopter. To close, we will cover the physics concepts which govern the behavior of our quadcopter and reflect back upon our experiences.

Contents

1 Introduction **3**

2 Components **3**

3 Mechanical Design **4**

4 Electrical Design **11**

5 Build **15**

6 Software **16**

7 Flight **22**

8 Conclusion **25**

9 References **26**

1 Introduction

As our society advances further into the technological era, engineering and physics skills become a necessity. Recognizing this, as future engineers, we wanted to put our skills to the test by constructing a quadcopter. The overarching goal of this project was to explore and understand the various aspects of engineering and physics that govern the process of creating a functional quadcopter.

2 Components

Below are all of the materials used to make the quadcopter.

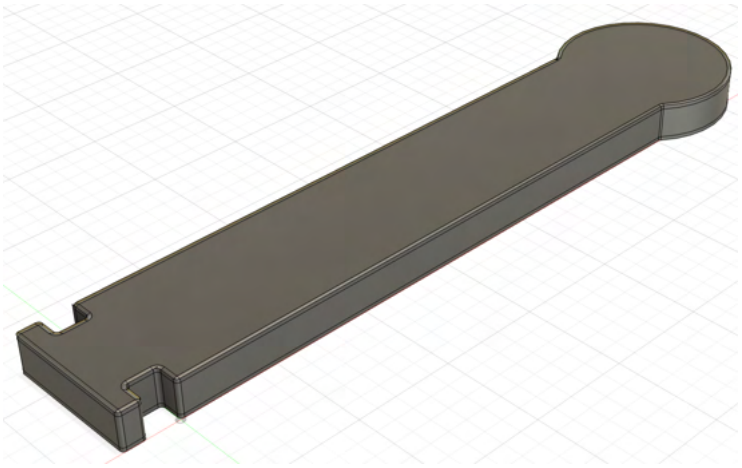
4x 1000KV Brushless Motor 13T
4x 40A RC Brushless Electric Speed Controller
4x 2 sets of Propellers
1x Power Distribution Board
1x FS-i6 6CH 2.4GHz RC Controller
1x FS-iA6B Receiver
Male & Female Bullet Connectors
Red & Black of Heat Shrink Tubing
1x Roll of Electrical Tape
1x HC-05 Wireless Bluetooth module
1x Adafruit Perma-Proto Board
3x 3.3k resistorx
2x 10k resistors
1x Arduino Uno
1x 3s lipo
1x Lipo Charger
1x Female XT-30 to Male XT-60 Adapter

Some notable tools used were wire strippers, a helping hand for soldering, solder wicks, a soldering iron, drills, a drillbit kit, a pin, and scissors.

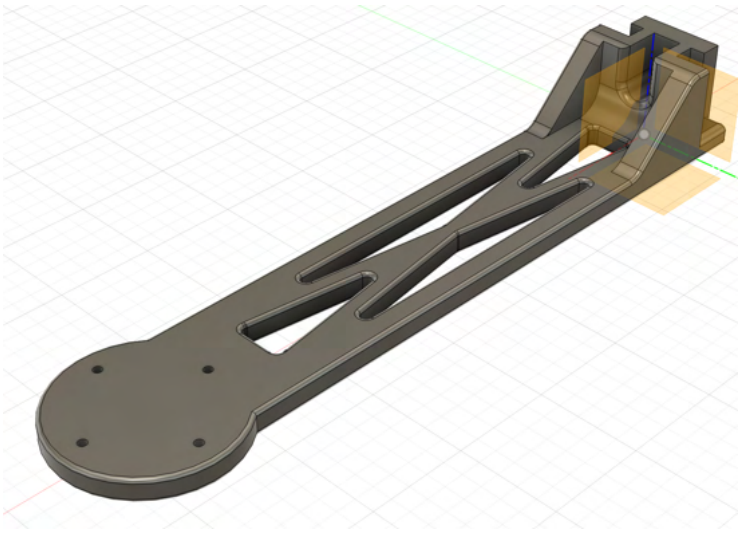
3 Mechanical Design

The design process for the drone arms considered several key factors: propeller size, motor placement, connection mechanism, and the overall mass of the arm.

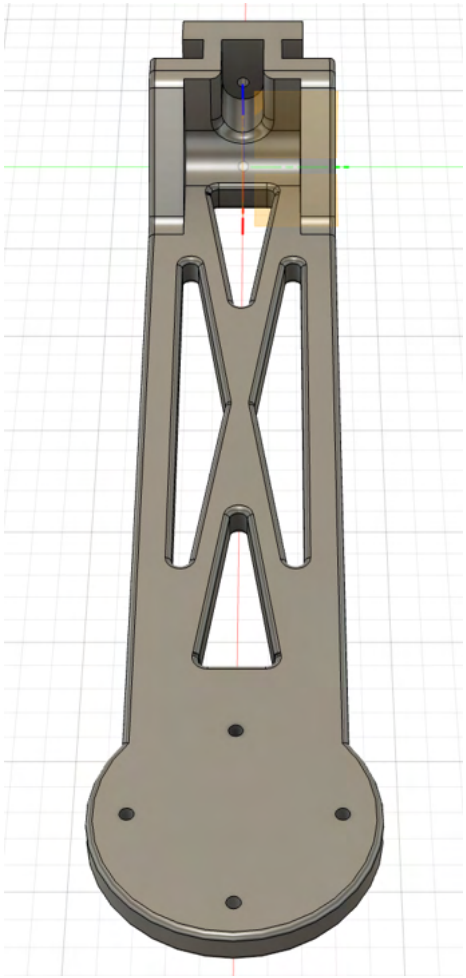
Our arm design was inspired by two main sources: a Mini Quad FPV Drone CAD model on OnShape [1] and MikeyB's FPV Drone on Thingiverse [2]. The Mini Quad model inspired our implementation of a friction-fit connection with horizontal screw holes, maximizing torque transfer while minimizing print time. MikeyB's design influenced our use of a thin, lightweight structure with an X-shaped cutout.



Our initial prototype (above) featured a friction-fit slot design but lacked firm connection support or access to screw holes. It was also too heavy, resulting in downward drooping caused by gravitational torque acting on the arm's center of mass.



The revised design improved structural stability, reduced weight, and included horizontal screw holes to enhance attachment. It also significantly reduced drooping. The arm is printed upside down to minimize overhangs during 3D printing. The central X-shape not only lightens the arm but also reduces print time and material usage.

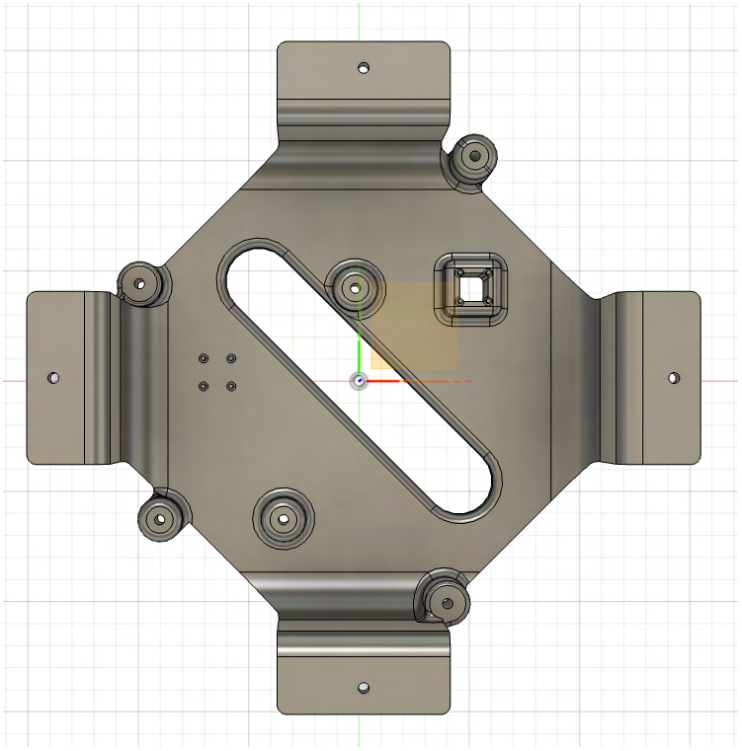


Furthermore, the motor holes, as seen above at the bottom of the image, along with the horizontal screw hole used to connect the arm to the body, as seen at the top of the image, were printed without a thread, however, it was later created by a bolt into the hole and pulling it out.

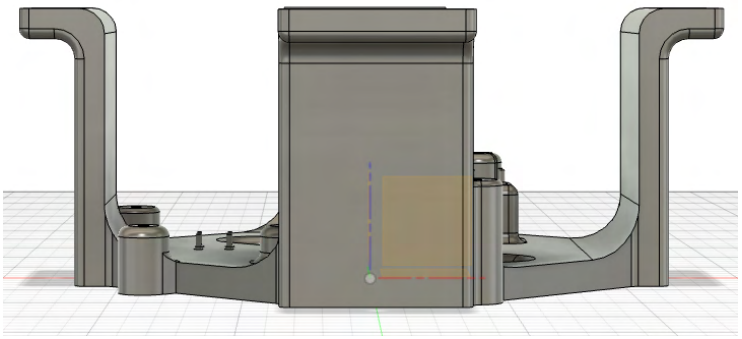
The shape of the center was custom-designed to accommodate the dimensions of key internal components: the Arduino UNO, power distribution board (PDB), flight control board, and arm connection points. The body consists of the undercarriage and the upper connection hub.



Figure 1: Final Arm design



The undercarriage houses the Arduino UNO and flight control board, which together serve as the drone's brain. Six circular holes were included to accommodate bolts for secure mounting. The four elevated pins on the right and four pins on the left are designed to hold foam padding: one side supports the flight control board (right), and the other secures the Arduino (left). Additionally, an elliptical cutout was implemented in the center to reduce material usage and print time.



As seen above, the undercarriage implements swan-neck style brackets that create a strong and materially efficient container for the flight control board and the Arduino. Additionally, the brackets are used to secure the undercarriage to the upper connection hub, which joins the entire build together.

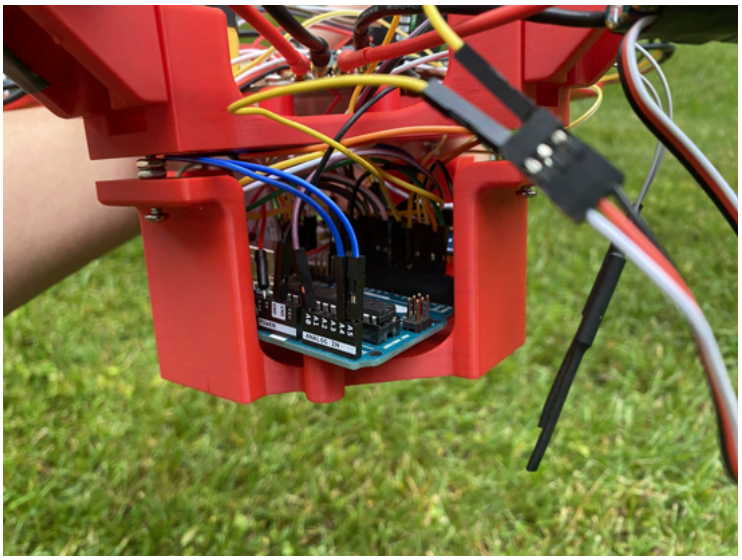
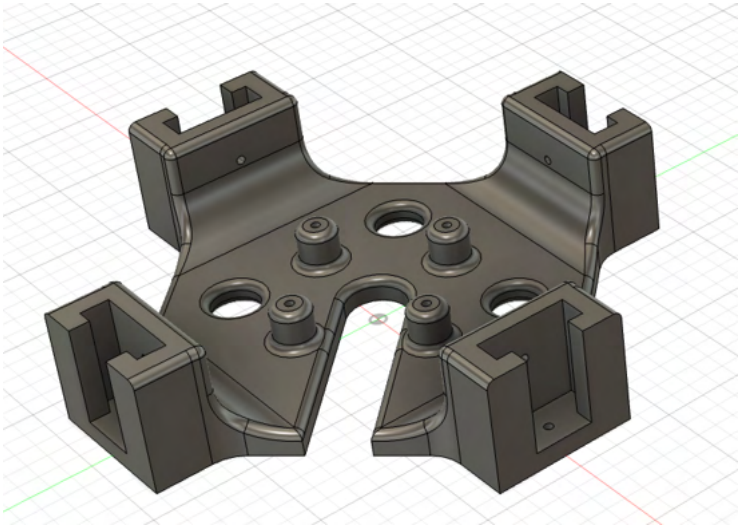


Figure 2: Final Undercarriage



The upper carriage is the most critical component of the drone's structure. It connects to the arms via T-shaped slots and screws, while also attaching to the undercarriage through screw holes located at the base of each T-slot. The four elevated circular holes in the center are used to mount the power distribution

board (PDB). Additionally, the remaining four cutouts serve as cable routing channels from the undercarriage, improving overall cable management.

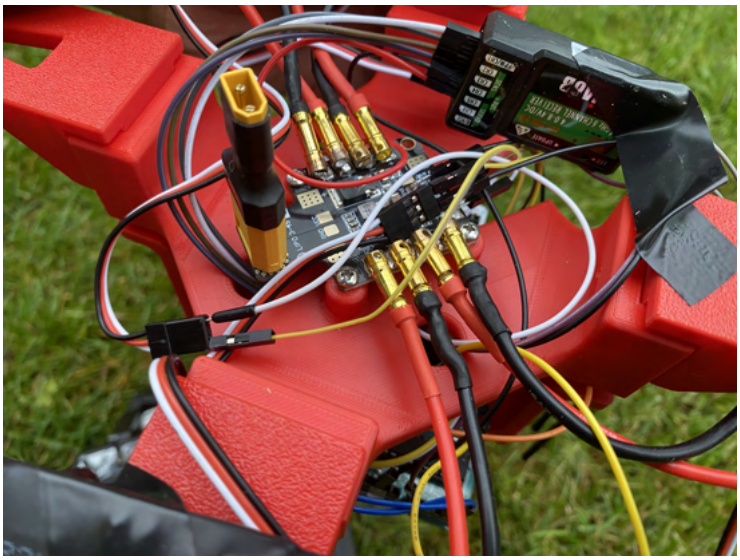


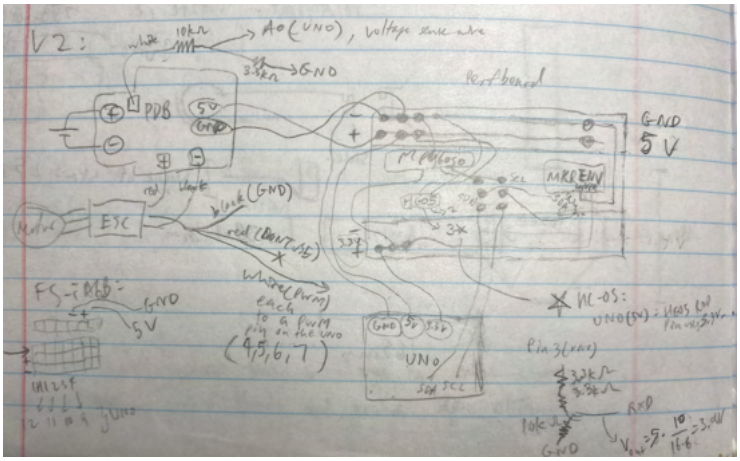
Figure 3: Final Upper Connection Hub



Figure 4: Final Completed Build

4 Electrical Design

Being relatively new to designing boards, we had to start with the fundamentals of circuit design, including power and communication.

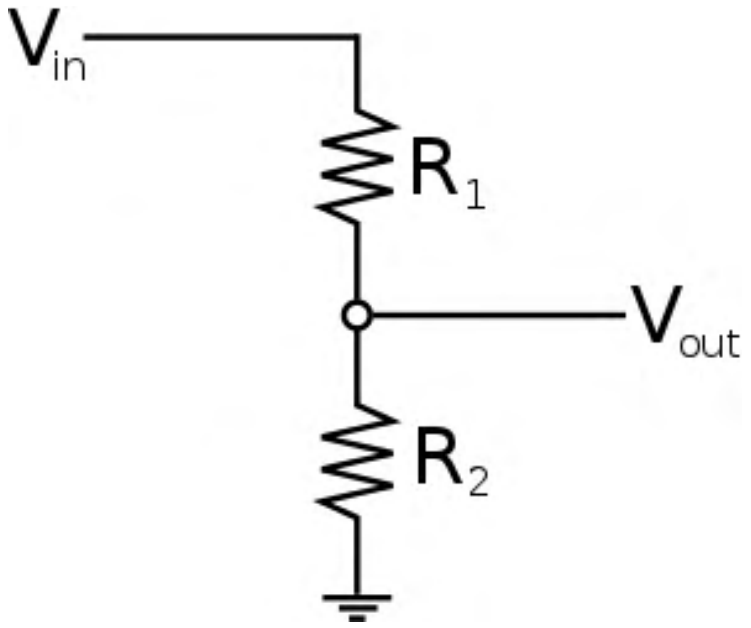


The first challenge was to safely deliver power to all components, including the electronic speed controllers or ESCs, Arduino Uno, HC-05 Transmission Module, MPU6050, and Arduino MKR ENV Shield. Above is a revised sketch of power distribution and communication.

Importantly, each ESC has 3 wires connected to its respective motor, which on its opposite side has 3 more outgoing wires; a thick red and black wire will be soldered to a positive and negative square on the power distribution board or PDB, respectively. The final outgoing wire has 3 female wire headers, one black, one white, and one red. Upon researching the purpose of each wire, the red wire would not be needed to connect to anything, as its purpose of powering other components will be unnecessary. As per the sketch, a wire will connect the 5V pad on the PDB to a rail on the board, from which the Uno and HC-05

will be able to get power from when we eventually disconnect the laptop from the Uno.

Initially, the Arduino Uno was powered through a direct USB connection to a laptop, which would then transfer 3.3V and 5V power and have a GND connection with the other 3 sensors. Upon further inspection, the HC-05 required a power range of 3.6V to 6V, which meant a 5V power supply was necessary. However, the RX pin on the sensor required a 3.3V power supply. This conundrum led to our exploration of ways to lower the voltage entering a certain wire or area.



We stumbled across the circuit shown above, known as a voltage divider. This circuit would be necessary to bring an incoming 5V power supply down to a usable 3.3V supply for the RX pin on the HC-05. To understand exactly how this voltage is divided, we can use Kirchhoff's Voltage Law and Ohm's Law

to analyze the circuit. R_1 and R_2 are in series with V_{in} on one end and GND ($V = 0$) on the other end. We can apply these laws over this wire to get:

$$V_{in} = IR_1 + IR_2$$

$$V_{out} = IR_2$$

We can now isolate I, equate the two equations, and solve for V_{out}:

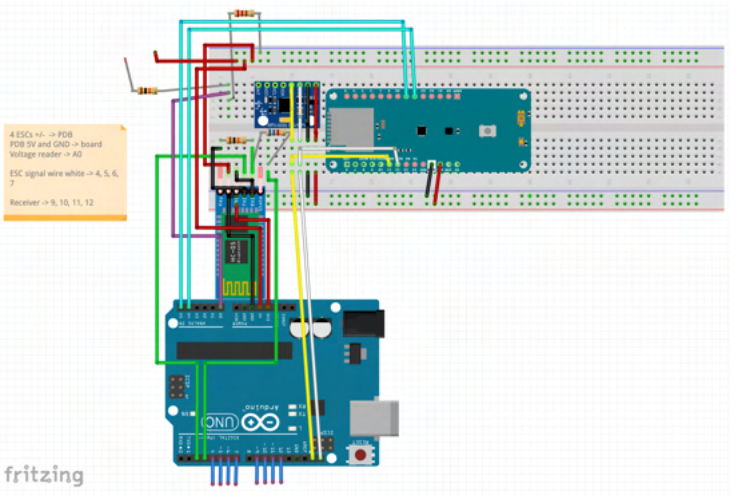
$$V_{in} = I(R_1 + R_2) \implies I = \frac{V_{in}}{R_1 + R_2}$$

$$V_{out} = IR_2 \implies I = \frac{V_{out}}{R_2}$$

$$\frac{V_{in}}{R_1 + R_2} = \frac{V_{out}}{R_2} \implies V_{out} = V_{in}(\frac{R_2}{R_1 + R_2})$$

With our 3.3k Ω and 10k Ω resistors, we could let $R_1 = 10000\Omega$ and $R_2 = 2 \times 3300\Omega = 6600\Omega$, which when $V_{in} = 5V$ yields $V_{out} = 5(\frac{10}{16.6}) = 3.01V$, which suffices as a voltage supply entering the RX pin.

To allow for communication between the ESCs, receiver, Uno, and sensors, we designated pins and used I2C Protocol. We let the signal wire (white female wire header) on each ESC connect to pins 4, 5, 6, and 7 on the Uno, respectively. The receiver contains 6 PWM (Pulse-Width Modulation) channels, where we can use 4 of them to connect to pins 9, 10, 11, and 12 on the Uno, allowing for a direct line of communication to each ESC from the Uno.



Above is an image of the final board design of the quadcopter using the Fritzing software. Only half of the breadboard is used due to size constraints of the compartment in our CAD, as well as the fact that our permanent prototyping board is the size of only half a breadboard.

After soldering the components, we used a drill to assemble each part together. We chose #4-40 screws of varying length, which were small enough to allow for holes to be created in parts of our CAD and available for us to use during this project.

5 Build

The process of acquiring all our materials and assembling them required a large amount of time. One particularly frustrating, time-consuming, somewhat hazardous, yet necessary portion of the build was soldering.

Soldering the bullet connectors to the PDB was by far the most difficult task of the physical construction of the quadcopter. When attempting to place the bullet connector onto a pad on the PDB, the previously liquefied solder would immediately harden as I took the soldering iron away to make space for the bullet connector. Eventually, we figured out that by carefully liquefying the solder while placing the bullet connector on it and then slowly taking away the soldering iron, we could make a decent connection.

We also needed to test how well our flight controller can control the 4 motors. To not accidentally supply the Uno with 2 power supplies, we noted that we could disconnect the 5V wire from the Uno, allowing the Lipo to only power the PDB and 4 motors, while the Uno's direct connection to the laptop will allow for uploading of code, Serial plotting & data, and a safe power supply for the rest of the electrical components.

6 Software

Being able to simultaneously power the motors and connect the Uno to a laptop allowed us to do PID (Proportional-Integral-Derivative) tuning.

```
#include <Wire.h>
#include <SoftwareSerial.h>
#include <Servo.h>
#include <MPU6050.h>
#include <Arduino_MKRENV.h>

// RC Receiver pins
uint8_t PIN_CH_ROLL     =  9;  // CH1 Roll
uint8_t PIN_CH_PITCH    = 10;  // CH2 Pitch
uint8_t PIN_CH_THROTTLE = 11;  // CH3 Throttle
uint8_t PIN_CH_YAW      = 12;  // CH4 Yaw

// ESC output pins
uint8_t PIN_ESC_FL = 4;  // front-left  (motor 1)
uint8_t PIN_ESC_FR = 5;  // front-right (motor 2)
uint8_t PIN_ESC_BL = 6;  // back-left   (motor 3)
uint8_t PIN_ESC_BR = 7;  // back-right  (motor 4)

Servo escFL, escFR, escBL, escBR;
MPU6050 mpu;
SoftwareSerial BTSerial(3, 2); // RX, TX

// Proportional
float Kp_rate_roll  = 0.90f;
float Kp_rate_pitch = 0.90f;
float Kp_rate_yaw   = 0.90f;

// Integrate
float Ki_roll = 0.002f;
float Ki_pitch = 0.002f;
float Ki_yaw = 0.002f;

float int_err_roll = 0;
float int_err_pitch = 0;
float int_err_yaw = 0;

// Derivative
float Kd_roll = 0.04f;
float Kd_pitch = 0.04f;
float Kd_yaw = 0.04f;

float prev_err_roll = 0;
float prev_err_pitch = 0;
float prev_err_yaw = 0;



// read
uint16_t readChannel(uint8_t pin) {
  return pulseIn(pin, HIGH, 30000);
}
```

```
52
53 void setup() {
54   Serial.begin(115200);
55   BTSerial.begin(9600);
56   Wire.begin();
57
58   mpu.initialize();
59   mpu.setFullScaleGyroRange(MPU6050_GYRO_FS_2000);
60   if (!mpu.testConnection()) {
61     Serial.println(F("MPU6050 connection failed"));
62     while (1) delay(10);
63   }
64
65   if (!ENV.begin()) {
66     Serial.println("Failed to initialize MKR ENV Shield");
67     while (1) delay(10);
68   }
69
70   Serial.println("All sensors initialized.");
71   BTSerial.println("BT OK: sensors initialized.");
72
73   escFL.attach(PIN_ESC_FL);
74   escFR.attach(PIN_ESC_FR);
75   escBL.attach(PIN_ESC_BL);
76   escBR.attach(PIN_ESC_BR);
77
78   escFL.writeMicroseconds(1000);
79   escFR.writeMicroseconds(1000);
80   escBL.writeMicroseconds(1000);
81   escBR.writeMicroseconds(1000);
82   delay(2000);
83   Serial.println("All ESCs armed.");
84 }
85
86 void loop() {
87   static long last_time = micros(); // isn't assigned to micros()
         after first use because it is static
88   long now = micros();
89   float dt = (now - last_time) / 1e6f;  // dt in seconds, convert
         from microseconds to seconds with 1e6
90   last_time = now; // reassign last_time
91
92   if (dt <= 0 || dt > 0.1f) dt = 0.01f; // safety
93
94   uint16_t pwm_roll     = readChannel(PIN_CH_ROLL);
95   uint16_t pwm_pitch    = readChannel(PIN_CH_PITCH);
96   uint16_t pwm_throttle = readChannel(PIN_CH_THROTTLE);
97   uint16_t pwm_yaw      = readChannel(PIN_CH_YAW);
98
99   // Map sticks to 1000-2000
100  float desired_rate_roll  = map(pwm_roll,     1000, 2000, -100,
         100);
101  float desired_rate_pitch = map(pwm_pitch,    1000, 2000, -100,
         100);
102  float desired_rate_yaw   = map(pwm_yaw,      1000, 2000, -100,
         100);
103
```

```
104   int16_t gx, gy, gz;
105   mpu.getRotation(&gx, &gy, &gz);
106   float rate_roll  =  (float)gy / 16.4f;
107   float rate_pitch =  (float)gx / 16.4f;
108   float rate_yaw   = (float)gz / 16.4f;
109
110   float err_roll  = desired_rate_roll  - rate_roll;
111   float err_pitch = desired_rate_pitch - rate_pitch;
112   float err_yaw   = desired_rate_yaw   - rate_yaw;
113
114   // Integrate
115   int_err_roll  += err_roll * dt;
116   int_err_pitch += err_pitch * dt;
117   int_err_yaw   += err_yaw * dt;
118
119   // Safeguard
120   int_err_roll  = constrain(int_err_roll, -50, 50);
121   int_err_pitch = constrain(int_err_pitch, -50, 50);
122   int_err_yaw   = constrain(int_err_yaw, -50, 50);
123
124   // Derivative
125   float d_err_roll  = (err_roll  - prev_err_roll)  / dt;
126   float d_err_pitch = (err_pitch - prev_err_pitch) / dt;
127   float d_err_yaw   = (err_yaw   - prev_err_yaw)   / dt;
128
129   prev_err_roll  = err_roll;
130   prev_err_pitch = err_pitch;
131   prev_err_yaw   = err_yaw;
132
133   // PID
134   float corr_roll  = Kp_rate_roll  * err_roll  + Ki_roll  *
          int_err_roll  + Kd_roll  * d_err_roll;
135   float corr_pitch = Kp_rate_pitch * err_pitch + Ki_pitch *
          int_err_pitch + Kd_pitch * d_err_pitch;
136   float corr_yaw   = Kp_rate_yaw   * err_yaw   + Ki_yaw   *
          int_err_yaw   + Kd_yaw   * d_err_yaw;
137
138   Serial.print(err_roll,1);
139     Serial.print("\t");
140   Serial.print(corr_roll,1);
141     Serial.print("\t");
142   Serial.print(desired_rate_roll,1);
143     Serial.print("\t");
144   Serial.print(err_pitch,1);
145     Serial.print("\t");
146   Serial.print(corr_pitch,1);
147     Serial.print("\t");
148   Serial.print(desired_rate_pitch,1);
149     Serial.print("\t");
150   Serial.print(err_yaw,1);
151     Serial.print("\t");
152   Serial.print(corr_yaw,1);
153     Serial.print("\t");
154   Serial.println(desired_rate_yaw,1);
155
156   int motorFL = pwm_throttle + corr_pitch + corr_roll - corr_yaw;
157   int motorFR = pwm_throttle + corr_pitch - corr_roll + corr_yaw;
```

```
  int motorBL = pwm_throttle - corr_pitch + corr_roll + corr_yaw;
  int motorBR = pwm_throttle - corr_pitch - corr_roll - corr_yaw;

  motorFL = constrain(motorFL, 1000, 2000);
  motorFR = constrain(motorFR, 1000, 2000);
  motorBL = constrain(motorBL, 1000, 2000);
  motorBR = constrain(motorBR, 1000, 2000);

  escFL.writeMicroseconds(motorFL);
  escFR.writeMicroseconds(motorFR);
  escBL.writeMicroseconds(motorBL);
  escBR.writeMicroseconds(motorBR);

  delay(10);
}
```

Listing 1: Flight Control Program

Above is the final code uploaded to the Uno. Certain parts of the code ended up unused, including the MKR ENV Shield and HC-05, due to flight complications that are discussed later. Additionally, online resources, including AI, were used to find the methods and organization needed to define PID tuning and reading channels from the receiver. The most challenging part of PID tuning, however, still lied in understanding how this process actually worked and could be implemented into code.

PID tuning reduces the error between the input into the RC controller and the detected roll, pitch, and yaw. Because our quadcopter never truly took flight, we were only able to do PID tuning while it remained stationary. Despite this, we were able to tune the response of the quadcopter on its 3 axes of rotation.

Mathematically, PID tuning is represented by

$$u(t) = K_p e(t) + K_i \int e(t)dt + K_d \frac{de}{dt}$$

where K_p, K_i, K_d are empirical constants, $u(t)$ represents corrected error, and

$e(t)$ represents error.

The first type of control is P (proportional) control. When given an input, for example, roll, that value is compared to the roll from the MPU6050 gyroscopic data. This difference is calculated and then multiplied by an empirically determined constant K_p, which is added to the corrected roll. This corrected roll is eventually added or subtracted to the throttle of each motor, depending on their orientation on the quadcopter.

The next type of control is I (Integral) control. As seen in the code, the variable `int_err_roll` gradually accumulates in value in response to long-term errors, preventing roll, pitch, and yaw from increasing or decreasing in value over time. This value is multiplied by a constant K_i to prevent overshooting the gradual error accumulation, which is measured and refined through testing. Finally, it is added to the corrected value like the proportional parameter.

The next type of control is D (Derivative) control. This parameter resembles the first term of a Taylor Series, helping reduce sudden error changes when it is multiplied by an empirically defined constant K_d and added to the corrected variable, as seen in the code.



With these 3 parameters in place, we tested moving the roll stick on the RC controller all the way right and then all the way left. The graph indicates that the real roll closely follows the roll input by the RC controller (Values 1, 2, and 3 are error, corrected error, and desired error, respectively).

7 Flight

The final step was to test the flight of the quadcopter. Initially, we observed 3 motors spinning clockwise and the last one spinning counterclockwise. We switched the wires around and plugged in the battery, but now the propellers of the quadcopter would fly off as soon as a certain amount of throttle was applied. We eventually successfully tightened the propellers, but despite PID tuning, adjusting the center of mass, checking the propellers and motors, and refining the code, the quadcopter flips over as soon as enough throttle is applied on the controller.

We believe an unequal amount of power is given to each motor upon giving the quadcopter some amount of throttle, which could have been caused by soldering, wiring issues, or faulty code. Another issue may be mapping the orientation of each motor from the code to the vehicle's real motor orientation, which may cause certain motors to get extra or insufficient throttle.

There are design considerations regarding quadcopters that were unknown to us prior to undertaking this project, including the orientation of the motors and propellers along with key concepts such as roll, pitch, lift, and yaw.

Propellers are crucial as they convert the rotational motion of the motors into lift by accelerating air downward.

Key aspects of propellers include

- **Blade Pitch:** The angle of the blades determines how much air is pushed downward.

- **Diameter:** Larger diameters sweep more air, producing more lift.
- **Rotation Direction:** Two propellers spin clockwise (CW), and two spin counterclockwise (CCW) to balance torque and enable yaw control.

Propellers must be oriented such that their leading edge is above their trailing edge.

The acceleration of air downward is called lift. Lift is the upward force exerted by the propellers of a quadcopter for it to, quite literally, lift off the ground. The lift generated by a single propeller can be given by the equation below:

$$L = C_L \cdot \frac{1}{2} \cdot \rho \cdot A \cdot v^2$$

Where L is the lift force (Newtons), C_L is the lift coefficient (depends on blade shape/pitch), ρ is the air density (kg/m^3), A is the swept area of the propeller (m^2), and v is the velocity of the air above the propeller (m/s). Furthermore, Lift is summative, so in the case of our device, $\Sigma L = L_1 + L_2 + L_3 + F_4$.

While lift generated by multiple propellers is additive, using several propellers in close proximity introduces additional challenges. Each spinning propeller generates not only lift but also a reactive torque on the frame, as explained by Newton's Third Law. If all four propellers were to spin in the same direction, the resulting net torque would cause the entire frame to spin uncontrollably.

To counteract the reactive torque generated by the propellers, we configured two to spin clockwise and the other two counterclockwise. This arrangement

balances the torques, resulting in a net torque of zero and stabilizing the drone. The resulting rotational imbalance, known as **yaw**, can be intentionally controlled to rotate or turn the quadcopter during flight. By adjusting the relative speeds of the clockwise and counterclockwise motors, controlled yaw motion enables turning and lateral movement.

Besides yaw, the quadcopter can also turn using **pitch** and **roll** controls. Pitch alters the speeds of the front and rear motors, causing rotation about the y-axis, while roll changes the speeds of the left and right motors, causing rotation about the x-axis. Yaw corresponds to rotation about the z-axis.

These rotational torques can be described mathematically as:

$$\tau_{\text{roll}} = (L_2 + L_3) - (L_1 + L_4)$$

$$\tau_{\text{pitch}} = (L_1 + L_2) - (L_3 + L_4)$$

$$\tau_{\text{yaw}} = k_T \left(\omega_2^2 + \omega_4^2 - \omega_1^2 - \omega_3^2\right)$$

where L_i denotes the lift force from motor i, ω_i the angular velocity of motor i, and k_T a constant relating motor speed squared to torque.

8 Conclusion

There are a couple of lessons that we have learned and can use in our future endeavors. Firstly, gaining a holistic understanding of an aspect of a project before diving into it will inevitably save time and effort. In hindsight, many hours were painstakingly wasted on soldering inefficiently, misunderstanding PID tuning, and forgetting an important connection on the prototyping board design. Another important note is to carefully check the connections between parts of a project; we had to obtain new adapters, Lipos, and types of wires to construct the quadcopter, which could have been time saved if we thoroughly accounted for these connections. Lastly, allowing extra time/buffers to complete a portion of a project should be generously applied, which would prevent cramming and stress. With all these insights now in our possession, future projects should be more streamlined and enjoyable to complete.

While the quadcopter did not ultimately achieve flight, we have gained invaluable experience and insight into the unexpected and expected challenges that come with fully designing and constructing such a vehicle.

Thanks for reading!

9 References

1. Onshape. *Onshape.com*, 2025, https://cad.onshape.com/documents/281222d51728d347853c14d0/w/1720634e578f5f454952f021/e/8387e60dd185aa2e25e73537. Accessed 16 June 2025.

2. Thingiverse.com. "3D Printed FPV Quadcopter the Crossfire by MikeyB." *Thingiverse*, 2025, https://www.thingiverse.com/thing:32281. Accessed 16 June 2025.